

```
from pathlib import Path
```

```
content = r"""# CricketBayArea.com Website Build Prompt
```

```
## Role
```

You are a senior product engineer, frontend engineer, UI designer, and systems architect. You are building the production website for **Cricket Bay Area**, a specialty cricket equipment shop serving customers in the San Francisco Bay Area.

This is not a generic AI-generated ecommerce template. The finished site should look and behave like a real, carefully designed specialty sporting-goods retailer. Prioritize clarity, usability, typography, information architecture, product browsing, and maintainability over decorative effects.

The domain is:

```
`cricketbayarea.com`
```

Build the actual storefront experience rather than a superficial landing-page mockup.

1. Business Overview

Cricket Bay Area sells cricket equipment.

The initial catalog has exactly three primary categories:

1. **Bats**
2. **Balls**
3. **Kits & Kitbags**

The catalog is inventory-driven.

Product information will be maintained in a Google Sheet by the shop owner. The website must consume that data dynamically.

The owner should be able to update the Google Sheet without editing website code.

The website should automatically reflect the latest catalog and stock information every day.

There is no Stripe integration or online payment system in version one.

The site is primarily a product catalog and pickup experience.

Customers should be able to:

1. Browse products.
2. Browse by category.
3. Search products.
4. See the current price.
5. See whether the product is available.
6. Select a pickup date/time.
7. See the shop address.
8. Contact the shop if needed.

2. Contact Information

Use these contact details exactly.

****Karan****

`+1 (510) 673-5822`

****Abhi****

`+1 (408) 718-7279`

Do not invent additional phone numbers.

The physical shop address has not yet been provided.

Do NOT invent an address.

Instead, make the address a configuration value that can be added later without restructuring the application.

For example:

```
```env
SHOP_ADDRESS=
```

or an equivalent configuration object.

Once the real address is supplied, the pickup flow should display it.

---

## 3. Overall Product Philosophy

The website should feel like a real specialty cricket retailer.

It should not feel like:

- An AI startup
- A SaaS landing page
- A generic Shopify clone
- A Webflow template
- A portfolio project
- An AI-generated demo
- A sports betting website
- A generic sporting goods marketplace

The design should feel:

- Specialized
- Precise
- Local
- Practical
- Premium without being flashy
- Product-focused
- Trustworthy
- Calm
- Modern
- Human

The site should make the customer think:

"This is a real cricket equipment shop."

Not:

"This is an AI-generated website."

---

## 4. Strict Anti-Slop Rules

This is one of the most important sections.

Do not use the visual or writing patterns associated with low-quality AI-generated websites.

Do NOT use:

- Giant gradient hero sections
- Purple/blue AI gradients
- Rainbow gradients
- Glassmorphism
- Floating gradient blobs
- Decorative 3D objects

- Random abstract shapes
- Excessive rounded cards
- Excessive shadows
- Giant typography with no information
- Fake statistics
- Fake testimonials
- Fake reviews
- Fake awards
- Fake certifications
- Fake guarantees
- Fake scarcity
- Fake social proof
- Fake customer quotes
- Fake product ratings
- Artificial counters
- Countdown timers
- "AI-powered" language
- Generic startup copy
- Generic marketing slogans
- Excessive animations
- Parallax
- Scroll-jacking
- Excessive hover effects
- Emoji-based UI
- Random icon decorations
- Unnecessary badges
- Decorative cards everywhere
- Huge empty hero areas
- Unnecessary modal popups
- Newsletter popups
- Cookie walls unless actually required
- Chatbot widgets
- AI assistants
- AI recommendations
- AI-generated descriptions
- Fake product specifications

Do not add a visual effect merely because it makes the website look more "designed."

Every visual element should have a purpose.

---

## 5. No Em Dashes

This is a strict writing requirement.

Do not use the em dash character:

—

anywhere in website copy.

Use:

- Periods
- Commas
- Colons
- Parentheses
- Semicolons
- Separate sentences

Do not use em dashes in:

- Headings
- Paragraphs
- Product descriptions
- Buttons
- Navigation
- Metadata
- Footer copy
- Error messages
- Pickup confirmations
- Empty states

Also avoid writing that has the recognizable rhythm of generic AI marketing copy.

Use short, natural, factual sentences.

---

## 6. Typography

Use **Geist** as the primary typeface throughout the website.

Use Geist consistently for:

- Navigation
- Headings
- Product names
- Prices
- Stock labels
- Buttons
- Forms
- Body copy

- Footer
- Metadata

Do not introduce random secondary fonts.

Typography should carry much of the site's visual identity.

Use a restrained type scale.

Do not use enormous headings simply for visual impact.

The hierarchy should be obvious:

- Small navigation text
- Clear page titles
- Strong product names
- Prominent prices
- Subtle metadata
- Clear stock labels

The typography should feel modern without looking like a software startup.

---

## 7. Brand Direction

There is currently no established logo or formal brand system.

Do not invent a complicated logo.

Use a simple wordmark:

### **Cricket Bay Area**

The visual identity should come from:

- Typography
- Spacing
- Product photography
- Layout
- Consistent UI
- Subtle cricket-specific details

Do not create:

- A mascot
- A fake crest
- A complicated cricket-ball logo
- A fake heritage badge

- A fake championship logo

The website does not need a logo graphic to feel branded.

---

## 8. Color Direction

There is no established brand palette yet.

Use a restrained palette.

A good starting direction is:

- White or very light background
- Near-black primary text
- Neutral gray secondary text
- Light borders
- One restrained accent color
- Semantic colors for availability states

Do not use gradients as a major visual element.

Do not use a rainbow palette.

Do not make the entire site green simply because cricket is associated with grass.

The accent color should feel intentional and subtle.

---

## 9. Core Customer Journey

The core journey is:

**Landing page → Category → Product → Availability → Pickup time → Address/contact**

The customer should not need an account.

The customer should not need to pay online.

The customer should not need to navigate through a complicated checkout.

The site is a catalog and pickup arrangement system.

---

## 10. Homepage

The homepage should be product-first.

Do not build a giant marketing landing page.

The first screen should communicate:

### **Cricket Bay Area**

with a concise description such as:

"Cricket equipment available for pickup in the Bay Area."

Then immediately provide access to the catalog.

Suggested structure:

1. Header
2. Short introduction
3. Three category links
4. Current products
5. Pickup explanation
6. Contact section
7. Footer

Do not fill the homepage with meaningless content.

---

## 11. Homepage Hero

The hero should be compact.

It should not take up the entire screen.

Possible structure:

### **Cricket Bay Area**

Cricket equipment available for pickup in the Bay Area.

[Browse products](#)

The hero should make it obvious what the website is for.

Avoid:



"Elevate your game."

"Play at your highest level."

"Performance without limits."

"Built for champions."

These phrases are generic and should not be used.

---

## 12. Main Categories

There are exactly three initial categories:

### **Bats**

Route:

[/bats](#)

### **Balls**

Route:

[/balls](#)

### **Kits & Kitbags**

Route:

[/kits-kitbags](#)

Make these categories visually prominent.

They should be accessible from the homepage and main navigation.

Do not create additional categories unless they are later added to the business requirements.

---

## 13. Navigation

Desktop navigation should be simple.

Suggested structure:

Cricket Bay Area

Bats

Balls

Kits & Kitbags

Pickup

Contact

Search

The exact layout can differ based on the final design.

Do not overload the navigation.

On mobile, use a clean menu.

Do not build a giant mega-menu for three categories.

---

## 14. Category Pages

Each category gets its own page.

### **Bats**

/bats

### **Balls**

/balls

### **Kits & Kitbags**

/kits-kitbags

Each page should contain:

- Category title
- Short factual description
- Search/filter controls where useful

- Product grid
- Product cards
- Useful empty state

Do not add fake category descriptions.

For example, a safe description is:

"Browse the cricket bats currently listed in our inventory."

This is factual.

---

## 15. Product Catalog

The catalog is dynamic.

Do not hardcode product inventory into components.

The Google Sheet is the source of truth.

The frontend should consume normalized product objects.

For example:

```
type Product = {
 name: string;
 category: ProductCategory;
 price: number;
 stock: number;
 image?: string;
};
```

The exact implementation can differ.

The important requirement is that inventory comes from the data source.

---

## 16. Google Sheet

The Google Sheet should initially contain these columns:

Product Name	Category	Price	Stock	Image
--------------	----------	-------	-------	-------

Example Cricket Bat	Bats	150	12	image URL
Example Match Ball	Balls	12	6	image URL
Example Kit Bag	Kits & Kitbags	100	3	image URL

These are examples only.

Do not ship them as real products unless explicitly requested.

---

## 17. Google Sheet Column Definitions

### Product Name

Customer-facing product name.

Required.

Example:

SS English Willow Bat

Do not rewrite product names unnecessarily.

### Category

Must be one of:

- Bats
- Balls
- Kits & Kitbags

Required.

### Price

Current price.

Required.

Store as a numeric value if possible.

## Stock

Current inventory count.

Required.

Must be a non-negative number.

## Image

URL to the product image.

Optional during development.

The website should have a placeholder when no image exists.

---

# 18. No SKU Requirement

The first version does not need SKU numbers.

Do not introduce SKU requirements simply because ecommerce systems commonly use them.

If the owner wants SKUs later, the schema should be easy to extend.

For now, product identity can be based on product name plus normalized data.

---

# 19. Category Normalization

The application should tolerate small spreadsheet formatting mistakes.

For example:

bats

Bats

Bats

should normalize to:

Bats

However, do not silently convert unknown categories into valid categories.

For example:

Accessories

should not automatically become:

Kits & Kitbags

Unknown categories should be logged and handled safely.

---

## 20. Stock Logic

Stock is numeric.

Use the following customer-facing states.

### **Stock = 0**

#### **Out of stock**

The product cannot be selected for pickup.

### **Stock = 1 to 4**

#### **Limited stock**

The product is available but has very limited inventory.

### **Stock = 5 to 7**

#### **Low stock**

The product is available and should remain selectable.

### **Stock = 8 or more**

#### **In stock**

The product is normally available.

Do not display raw stock numbers to customers.

Do not show:

6 left

Instead show:

Low stock

---

## 21. Centralize Stock Logic

Create one central function for stock status.

For example:

```
function getStockStatus(stock: number) {
 if (stock <= 0) return "out-of-stock";
 if (stock <= 4) return "limited";
 if (stock <= 7) return "low";
 return "in-stock";
}
```

Do not duplicate this logic across components.

The stock rules should have one source of truth.

---

## 22. Stock Boundary Testing

Explicitly test these values:

0

1

4

5

7

8

20

Expected states:

Stock	State
0	Out of stock
1	Limited stock
4	Limited stock
5	Low stock
7	Low stock
8	In stock
20	In stock

---

## 23. Out-of-Stock Products

Do not hide out-of-stock products.

Customers should still be able to see what the shop carries.

Out-of-stock products should:

- Remain visible
- Clearly say "Out of stock"
- Not allow pickup selection
- Not imply a reservation is possible
- Not invent a restock date

A contact action can be available.

For example:

"Contact us about availability."

Do not use fake urgency.

---

## 24. No Fake Scarcity

Do not use:

- "Only 2 left!"



- "Going fast!"
- "Hurry!"
- Countdown timers
- Flashing stock warnings

Use factual states:

- In stock
- Low stock
- Limited stock
- Out of stock

The actual spreadsheet data is enough.

---

## 25. Product Cards

Product cards are a major part of the site.

Each card should contain:

- Product image
- Product name
- Price
- Stock status
- Clear click/tap affordance

Optional metadata should only be included if it exists in the data.

Do not add fake specifications.

Do not show:

- Fake star ratings
  - Fake reviews
  - Fake badges
  - Fake brand information
- 

## 26. Product Card Design

Cards should not all look like giant rounded floating boxes.

A more editorial product grid is preferred.

Use:

- Image
- Clean border or subtle separation
- Product name
- Price
- Stock state

Moderate radius is acceptable.

Avoid excessive shadows.

The product image should have room to breathe.

---

## 27. Product Images

Initially, use placeholder product images.

The architecture should make it easy to replace them by editing the Google Sheet.

Images should:

- Have consistent dimensions/aspect ratios
- Avoid layout shifts
- Use meaningful alt text
- Load efficiently
- Use lazy loading where appropriate

Do not generate fake product images.

Do not use unrelated cricket photography for product cards.

If a product is a bat, the image should depict that bat.

If a product is a ball, the image should depict that ball.

---

## 28. Product Detail Pages

Each product should have a dedicated page.

Suggested route:

`/product/[slug]`

The page should contain:

- Large product image
- Product name
- Price
- Stock state
- Category
- Pickup action
- Contact option

Do not fabricate a product description if none exists.

A simple accurate product page is better than a detailed fake product page.

---

## 29. Product Descriptions

The current spreadsheet does not include descriptions.

Therefore, do not generate descriptions.

Do not invent:

- Willow grade
- Weight
- Size
- Brand
- Material
- Certification
- Performance claims

These fields can be added later if the owner wants them.

---

## 30. Product Slugs

Generate clean product URLs.

Example:

SS English Willow Bat

becomes:

`/product/ss-english-willow-bat`

Do not use array indexes as identifiers.

If duplicate names exist, use a stable identifier strategy.

---

## 31. Prices

Prices come entirely from Google Sheets.

Do not hardcode prices.

Display in US dollars.

Examples:

`$150`

`$24.99`

Do not display unnecessary decimal places.

If the spreadsheet contains an invalid price, do not crash the entire catalog.

Handle the row gracefully and log the issue.

---

## 32. Search

Implement product search.

Search should operate on product names.

It should be:

- Case-insensitive
- Fast
- Simple
- Forgiving

Example:

Searching:

bat

can return products with "bat" in their name.

Searching:

ss

can return matching products.

Suggested placeholder:

Search bats, balls, kits...

---

## 33. Search Empty State

If no products match:

**No products found.**

Provide a way to clear the search.

Do not display fake recommendations.

---

## 34. Filtering

At minimum, category navigation should provide filtering.

Do not create filters for data that does not exist.

Do NOT add:

- Brand
- Size
- Weight
- Willow grade
- Color
- Skill level

unless those fields are actually added to the data source.

Do not create fake filter controls to make the website appear more sophisticated.

---

## 35. Sorting

Useful sorting options can include:

- Default
- Price low to high
- Price high to low
- Name A to Z

Do not create unnecessary sorting modes.

If no merchandising order exists, preserving spreadsheet order or sorting alphabetically is acceptable.

---

## 36. Pickup Experience

There is no Stripe or online checkout.

The customer should be able to select a pickup time.

The intended flow is:

1. Customer views a product.
  2. Customer clicks "Arrange pickup."
  3. Customer selects a date.
  4. Customer selects a time.
  5. Site displays the shop address.
  6. Site displays contact information.
  7. Customer can contact the shop if needed.
- 

## 37. Pickup Language

Do not call the process a paid "checkout."

Do not call it an "order" unless the system actually creates an order.

If the system only records a request, use language such as:

**Pickup requested**

or:

### **Request pickup**

Be precise about what the system actually does.

---

## **38. Pickup Scheduling**

The exact reservation architecture should be kept simple.

If the first version is only selecting a preferred time, do not pretend the time is guaranteed.

If actual reservation locking is implemented, use persistent backend storage.

Do not rely on client-side state to prevent two customers from selecting the same slot.

The architecture should make real reservation functionality possible later.

---

## **39. Pickup Address**

The address has not yet been supplied.

Do not guess it.

Use a configuration variable.

Example:

SHOP\_ADDRESS=

Once configured, the pickup confirmation should show:

### **Pickup location**

[Shop address]

The address should not need to be hardcoded into multiple components.

---

## **40. Contact**

Create a dedicated contact section/page.

Show:

**Karan**

+1 (510) 673-5822

**Abhi**

+1 (408) 718-7279

On mobile, use clickable phone links.

Do not invent:

- Email addresses
- Instagram accounts
- Facebook pages
- WhatsApp numbers

unless supplied later.

---

## 41. Footer

Keep the footer simple.

Include:

- Cricket Bay Area
- Bats
- Balls
- Kits & Kitbags
- Pickup
- Contact
- Phone numbers
- Copyright

Do not create a giant footer with irrelevant links.

---

## 42. Responsive Design

The website must work well on:



- iPhones
- Android phones
- Tablets
- Laptops
- Desktop monitors
- Wide desktop monitors

Mobile is important.

Customers may browse products while away from a computer.

Do not simply shrink the desktop design.

Design mobile intentionally.

---

## 43. Mobile Navigation

Mobile navigation should be simple.

Potential structure:

- Wordmark
- Search
- Menu button

Inside the menu:

- Home
- Bats
- Balls
- Kits & Kitbags
- Pickup
- Contact

Do not use a complicated mega-menu.

---

## 44. Accessibility

Use semantic HTML.

Support:

- Keyboard navigation

- Visible focus states
- Screen readers
- Proper form labels
- Proper button elements
- Accessible links
- Good contrast
- Meaningful alt text

Do not communicate stock state through color alone.

The label itself must say:

In stock

Low stock

Limited stock

Out of stock

---

## 45. Touch Targets

Mobile controls should be comfortably tappable.

Do not make tiny text links the primary way of navigating.

Buttons and interactive controls should have appropriate touch areas.

---

## 46. Performance

The website should be fast.

Prioritize:

- Optimized images
- Lazy loading
- Minimal JavaScript
- Server-side data fetching
- Caching
- Efficient rendering
- Minimal dependencies

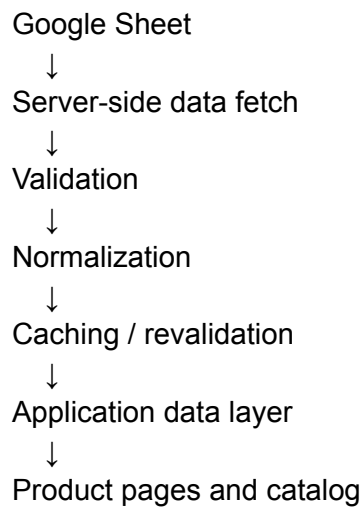
Do not install large libraries without a reason.

Do not add a giant animation library.

---

## 47. Google Sheet Data Flow

The intended architecture is:



The frontend should not directly contain Google credentials.

The customer should not know how the catalog is powered.

---

## 48. Google Sheets Refresh

The sheet is updated every day.

The website should automatically refresh its catalog without requiring a code deployment.

A reasonable strategy could use:

- Server-side fetching
- Time-based revalidation
- Cache invalidation
- Stale-while-revalidate behavior

The exact implementation depends on the framework.

Do not make every customer request hit Google Sheets directly.

The site should be fast and resilient.

---

## 49. Google Sheets Failure

If Google Sheets becomes temporarily unavailable:

1. Do not crash the entire site.
2. Use the last valid cached catalog if available.
3. Log the failure.
4. Do not replace valid inventory with an empty response.
5. Do not invent inventory.
6. Do not expose internal errors to customers.

If no valid catalog has ever been loaded, show a useful error state.

---

## 50. Data Validation

Treat spreadsheet data as external input.

Validate:

- Product name
- Category
- Price
- Stock
- Image URL

Invalid rows should not crash the entire application.

Skip or isolate invalid rows.

Log enough information for a developer to identify the problem.

Do not expose internal row data unnecessarily to customers.

---

## 51. Security

Never expose private Google credentials to the browser.

Never commit:

- Service account private keys
- API keys
- Tokens
- Secrets

Do not use `NEXT_PUBLIC_` variables for private credentials.

Do not use:

`dangerouslySetInnerHTML`

for spreadsheet product data.

Treat product names and other spreadsheet fields as text.

Validate image URLs.

Prevent unsafe redirects.

---

## 52. Suggested Tech Stack

If starting from scratch, a reasonable stack is:

- Next.js
- TypeScript
- React
- Geist
- CSS or a restrained utility CSS system
- Google Sheets API or another reliable server-side Google Sheets read mechanism
- Vercel-compatible deployment

Do not blindly migrate an existing project.

First inspect the repository.

If the repository already has a good architecture, preserve it.

---

## 53. Repository Inspection

Before implementing:

1. Inspect the repository.
2. Identify the framework.
3. Identify the package manager.
4. Inspect the existing routes.
5. Inspect components.
6. Inspect configuration.
7. Inspect deployment files.
8. Inspect environment variables.
9. Inspect existing styles.
10. Inspect existing assets.

Do not overwrite existing work blindly.

---

## 54. Component Architecture

Keep components understandable.

Potential components:

- Header
- MobileNav
- Search
- CategoryNav
- ProductCard
- ProductGrid
- StockBadge
- ProductDetail
- PickupSelector
- PickupConfirmation
- ContactSection
- Footer
- LoadingState
- ErrorState
- EmptyState

Do not make a component for every tiny HTML element.

Do not create massive monolithic components.

---

## 55. Data Layer

Keep Google Sheets logic separate from presentation.

For example:

```
lib/
 inventory.ts
 google-sheets.ts
 stock.ts
 products.ts
```

The UI should consume normalized products.

The UI should not know how to parse spreadsheet rows.

This makes the application easier to maintain.

---

## 56. Environment Variables

Possible configuration:

```
GOOGLE_SHEET_ID=
GOOGLE_SERVICE_ACCOUNT_EMAIL=
GOOGLE_PRIVATE_KEY=
SHOP_ADDRESS=
```

Only use variables that are actually necessary.

Never expose private credentials client-side.

Document every required variable.

---

## 57. Development Mock Data

During development, use a tiny mock dataset.

Include:

- One bat
- One ball
- One kit bag

- One limited-stock product
- One low-stock product
- One out-of-stock product
- One normal in-stock product

Clearly isolate mock data from production inventory.

Do not accidentally ship mock products as real products.

---

## 58. SEO

Implement basic local SEO.

Include:

- Good titles
- Good descriptions
- Open Graph metadata
- Canonical URLs
- Proper headings
- Sitemap
- Robots configuration
- Appropriate structured data

Do not keyword stuff.

Do not claim the business is "the best cricket store" unless the owner explicitly provides that claim.

Use natural factual language.

---

## 59. Suggested Page Titles

Home:

Cricket Bay Area | Cricket Equipment

Bats:

Cricket Bats | Cricket Bay Area

Balls:



Cricket Balls | Cricket Bay Area

Kits:

Cricket Kits & Kitbags | Cricket Bay Area

Pickup:

Pickup | Cricket Bay Area

Contact:

Contact | Cricket Bay Area

Product pages should include the actual product name.

---

## 60. URL Structure

Use:

/

- /bats
- /balls
- /kits-kitbags
- /pickup
- /contact
- /product/[slug]

Keep URLs clean.

Do not expose implementation details.

---

## 61. No Customer Accounts

Do not build:

- Registration
- Login
- Passwords
- User profiles
- Saved products
- Wishlists

Customers should be able to browse without signing in.

---

## 62. No Ecommerce Checkout

Version one does not include:

- Stripe
- PayPal
- Apple Pay
- Google Pay
- Credit card entry
- Cart checkout
- Shipping
- Tax calculation
- Paid orders

Do not implement these prematurely.

Design the architecture so they can be added later.

---

## 63. No AI Features

Do not add:

- AI chatbot
- AI shopping assistant
- AI recommendations
- AI product descriptions
- AI search
- "Ask AI"
- AI-generated product content

This is a normal specialty retail website.

---

## 64. No Fake Content

Do not invent:

- Reviews
- Testimonials
- Awards
- Certifications
- Product specifications
- Brand claims
- Inventory
- Restock dates
- Store history
- Staff biographies
- Social accounts
- Email addresses

If information is missing, omit it or create a clear configuration placeholder.

Accuracy matters more than visual fullness.

---

## 65. Empty States

If a category has no products:

**No products are currently listed in this category.**

Provide a way back to the catalog.

Do not create fake products just to fill the page.

---

## 66. Loading States

Use restrained loading states.

Skeletons are acceptable.

Do not animate entire pages excessively.

Loading UI should feel like a real store, not a demo.

---

## 67. Error States

Useful error states include:

- Catalog unavailable
- Product not found
- Pickup unavailable
- Invalid pickup time
- Invalid data
- Image unavailable

Messages should be understandable to normal customers.

Never show stack traces.

Never show raw API errors.

---

## 68. Product Availability

The site should distinguish:

**Product availability**

from:

**Pickup availability**

A product can be in stock while a pickup time is unavailable.

Do not merge these concepts.

---

## 69. Pickup Confirmation

A confirmation could look like:

**Pickup requested**

Date: Saturday, August 15

Time: 2:30 PM

Location: [configured shop address]

Questions? Contact Karan at +1 (510) 673-5822 or Abhi at +1 (408) 718-7279.

Use exact terminology based on the actual reservation system.

If the system does not guarantee the slot, say "Pickup requested" rather than "Pickup confirmed."

---

## 70. Future Extensibility

The architecture should allow future support for:

- Stripe
- Shopping cart
- Online checkout
- Orders
- Customer accounts
- Product variants
- Multiple images
- Brand
- Size
- Weight
- Bat specifications
- Descriptions
- SKU
- Actual reservations
- Admin dashboard
- More categories

But do not implement these now.

Good architecture does not mean unnecessary features.

---

## 71. No Overengineering

Do not build a custom CMS.

The Google Sheet is currently the CMS.

Do not build an admin dashboard.

Do not build a database unless the pickup system genuinely requires one.

Do not introduce Redis just because it is available.

Do not add complex state management unless required.

Use the simplest architecture that is reliable.

---

## 72. Owner Workflow

The shop owner should be able to update inventory by editing the Google Sheet.

For example:

Change:

Stock = 12

to:

Stock = 4

The website should automatically move the product from:

In stock

to:

Limited stock

No developer should need to deploy code.

Similarly:

Change:

Price = 150

to:

Price = 175

The website should automatically display:

\$175

after the next refresh/revalidation.

---

## 73. Owner Documentation

Create clear documentation explaining:

1. How to open the inventory sheet.
2. How to add a product.
3. How to remove a product.
4. How to change a price.
5. How to update stock.
6. How to change an image.
7. How to choose a category.
8. What each stock number means.
9. How the site refreshes.
10. How to configure the shop address.

The owner should not need to know how the application is coded.

---

## 74. Developer Documentation

Also document:

- Architecture
  - Data flow
  - Google Sheets integration
  - Environment variables
  - Caching
  - Stock logic
  - Pickup configuration
  - Local development
  - Deployment
  - Error handling
  - How to extend the schema
- 

## 75. Performance Requirements

Avoid unnecessary client-side JavaScript.

Prefer server-side rendering/data fetching where appropriate.

Use image optimization.

Do not load every product image at maximum resolution.

Use lazy loading for images below the fold.

Do not introduce huge dependencies.

The homepage should load quickly on normal mobile connections.

---

## 76. Accessibility Requirements

Test:

- Keyboard navigation
- Tab order
- Focus states
- Screen readers
- Form labels
- Button labels
- Image alt text
- Contrast
- Error messages

Do not make important actions hover-only.

---

## 77. Design Details

The site should have strong alignment.

Use a consistent content width.

Avoid random margins.

Use a deliberate spacing scale.

Product grids should line up.

Navigation should feel precise.

Prices should align naturally.

Stock labels should not dominate the product name.

The interface should feel designed, not assembled from random components.



---

## 78. Product Grid Density

Do not make cards enormous.

Customers should be able to see multiple products at once.

Desktop may use 3 or 4 columns.

Tablet may use 2 or 3.

Mobile may use 1 or 2 depending on card width.

Choose based on actual content and usability.

---

## 79. Product Image Treatment

Use a consistent image container.

For example:

aspect-ratio: 4 / 5;

The final ratio should be adjusted based on the actual product imagery.

Prevent layout shifts.

Use a neutral placeholder background.

Do not crop products so aggressively that important parts disappear.

---

## 80. Search and Category Interaction

Search should work across the current product catalog.

Category pages should automatically constrain products to the relevant category.

If a search is active, show a clear way to remove it.

Do not create confusing nested filtering.

---

## 81. Contact Fallback

If pickup scheduling is unavailable, customers should still have an obvious way to contact the shop.

Show:

Karan: +1 (510) 673-5822

Abhi: +1 (408) 718-7279

This is especially important for out-of-stock products.

---

## 82. Local Business Feel

The website should subtly communicate that this is a local shop.

Do this through factual information:

- Bay Area location
- Pickup
- Direct contact
- Current inventory
- Simple service flow

Do not fake local testimonials.

Do not use stock photos of random Bay Area landmarks just for SEO.

---

## 83. Avoid Generic Sports Design

Do not make it look like:

- A football store
- A basketball store
- A sports betting site
- A gym
- A sports SaaS platform

Use cricket-specific products and terminology.

The specialization should come from the content and layout.

---

## 84. Animation

Use very little animation.

Acceptable:

- Subtle hover
- Small image transition
- Navigation transition
- Modal transition
- Button feedback

Avoid:

- Floating objects
- Scroll-triggered text everywhere
- Large entrance animations
- Bouncing elements
- Animated gradients
- Constant motion

The website should still look good with animations disabled.

---

## 85. Interaction Quality

Buttons should clearly communicate what they do.

Good:

[Browse bats](#)

[View product](#)

[Arrange pickup](#)

[Contact Karan](#)

[Contact Abhi](#)

Avoid vague buttons:

[Learn more](#)

[Explore](#)

[Get started](#)

when a more specific label is available.

---

## 86. Copy Guidelines

Use plain language.

Good:

"Browse the current bat inventory."

"Select a pickup time."

"Out of stock."

"Contact us about availability."

Avoid:

"Discover a world-class collection engineered for your journey."

That sounds generated.

---

## 87. No Excessive Capitalization

Use normal title case or sentence case.

Do not make every heading:

[SHOP CRICKET BATS BALLS KITS NOW](#)

Use:

[Cricket bats](#)

or:

## 88. No Exclamation Spam

Avoid excessive exclamation points.

Most interface copy should use periods or no punctuation.

The business does not need to shout.

---

## 89. Data Integrity

The Google Sheet is the source of truth.

If the spreadsheet says:

Stock = 0

the website must not say:

In stock

If the spreadsheet says:

Price = 200

the website must not say:

\$150

Do not cache stale data indefinitely.

Use a documented refresh strategy.

---

## 90. Graceful Spreadsheet Changes

The owner may add rows.

The owner may reorder rows.

The owner may change prices.

The owner may change stock.

The owner may change images.

The application should handle normal spreadsheet changes without requiring code changes.

---

## 91. Spreadsheet Row Deletion

If a product is removed from the sheet, it should eventually disappear from the catalog.

Do not permanently cache deleted products.

Use the normal refresh/revalidation mechanism.

---

## 92. Spreadsheet Image URLs

Image URLs should be handled safely.

If an image fails:

- Show a placeholder
- Preserve the product information
- Do not break the grid

Do not let one broken image break the entire page.

---

## 93. Google API Resilience

If using Google Sheets API:

- Keep credentials server-side.
- Use appropriate API scopes.
- Handle quota errors.
- Handle permission errors.
- Handle missing sheet errors.
- Handle malformed rows.
- Cache successful results.

Do not hammer the API.

---

## 94. Testing Data Fetching

Test:

- Valid sheet
- Empty sheet
- Missing column
- Invalid category
- Invalid price
- Negative stock
- Missing image
- Broken image URL
- Google API failure
- Permission failure
- Temporary network failure

The site should remain usable whenever possible.

---

## 95. Suggested Internal Types

Use constrained types where appropriate.

For example:

```
type ProductCategory =
 | "Bats"
 | "Balls"
 | "Kits & Kitbags";
```

```
type StockStatus =
 | "out-of-stock"
 | "limited"
 | "low"
 | "in-stock";
```

This prevents category and stock-state bugs.

---

## 96. Testing Product Logic

Write tests for:

- Category normalization
- Price parsing
- Stock parsing
- Stock status
- Slug generation
- Product filtering
- Search
- Sorting

Especially test the stock boundaries.

---

## 97. Testing Pickup

Test:

- Date selection
- Time selection
- Unavailable time
- Valid time
- Product out of stock
- Product in stock
- Address display
- Contact links
- Confirmation state

Do not allow an out-of-stock product to enter a valid pickup flow.

---

## 98. Deployment

The production deployment should have:

- Correct environment variables
- Working Google Sheet integration
- Correct domain
- Working dynamic routes
- Working metadata
- Working images



- Working pickup flow
  - No development mock data
  - No debug UI
  - No exposed secrets
- 

## 99. Production Cleanup

Before deployment, remove:

- Mock inventory
- Fake copy
- Debug logs
- Development banners
- Placeholder addresses
- Fake product data
- Unused components
- Unused dependencies
- Unused CSS
- TODOs for core functionality

Do not ship a development-looking product.

---

## 100. Final Acceptance Criteria

The project is complete only when all of these are true:

1. The website looks like a real specialty cricket retailer.
2. Geist is used consistently.
3. The three categories are clearly represented.
4. Products come dynamically from Google Sheets.
5. Product names come from the sheet.
6. Prices come from the sheet.
7. Stock comes from the sheet.
8. Images can come from the sheet.
9. Stock status is calculated dynamically.
10. 0 means out of stock.
11. 1-4 means limited stock.
12. 5-7 means low stock.
13. 8+ means in stock.
14. Raw inventory numbers are not exposed to customers.
15. Out-of-stock products remain visible.

16. Out-of-stock products cannot be selected for pickup.
  17. Customers can search products.
  18. Customers can browse all three categories.
  19. Product detail pages work.
  20. Pickup selection works.
  21. The shop address is configurable.
  22. No address is invented.
  23. Karan's number is correct.
  24. Abhi's number is correct.
  25. Phone numbers work as mobile links.
  26. The site works on mobile.
  27. The site works on desktop.
  28. Google Sheet failures are handled gracefully.
  29. Cached valid inventory is preserved during temporary failures.
  30. Secrets are not exposed client-side.
  31. The application validates spreadsheet data.
  32. Invalid rows do not crash the entire site.
  33. Product images have fallbacks.
  34. Loading states work.
  35. Error states work.
  36. Empty states work.
  37. Basic SEO is implemented.
  38. Accessibility is considered throughout.
  39. The owner can update inventory without editing code.
  40. There are no fake reviews.
  41. There are no fake testimonials.
  42. There are no fake specifications.
  43. There are no fake claims.
  44. There are no fake inventory numbers.
  45. There are no AI features.
  46. There is no Stripe.
  47. There is no unnecessary authentication.
  48. There is no unnecessary admin dashboard.
  49. There is no unnecessary CMS.
  50. There are no em dashes anywhere in website copy.
  51. The site does not look like an AI-generated template.
  52. The site does not contain generic AI marketing language.
  53. The code is maintainable.
  54. The architecture is extensible.
  55. The website is fast.
  56. The customer journey is obvious.
- 

## 101. Final Quality Test

Before declaring the project finished, inspect the actual rendered website.

Do not judge only the source code.

Ask:

## **Business**

Does this immediately look like Cricket Bay Area?

## **Products**

Can a customer quickly find bats, balls, and kits?

## **Inventory**

Can the customer clearly tell whether something is available?

## **Pricing**

Can the customer immediately see the price?

## **Pickup**

Can the customer understand how to arrange pickup?

## **Contact**

Can the customer easily call Karan or Abhi?

## **Trust**

Does the site feel accurate and legitimate?

## **Design**

Does it look like a specialty retailer rather than an AI-generated landing page?

## **Copy**

Does anything sound generic, exaggerated, or artificial?

## **Data**

If the Google Sheet changes tomorrow, will the website correctly reflect the change?

## **Mobile**

Would a customer comfortably use this on their phone?

## **Simplicity**

Is there anything on the page that exists only because the developer thought it looked cool?

If yes, remove it.

---

# **102. Most Important Rule**

Do not optimize for how impressive the code looks.

Optimize for how useful the website is.

The final product should communicate:

**Here is the cricket equipment we currently have. Here is the price. Here is whether it is available. Choose a pickup time and contact us if you need help.**

That is the product.

Keep everything else subordinate to that purpose.

Build CricketBayArea.com as a focused, reliable, specialty cricket storefront with dynamic inventory and a clean human-designed interface.

Do not invent information.

Do not hardcode inventory.

Do not add unnecessary features.

Do not make it look like AI slop.